

ParkingVE

Documento Maestro Técnico v2.0

Guía oficial de desarrollo para Antigravity

Proyecto: ParkingVE

Tipo: PWA SaaS para gestión de estacionamientos pequeños

Backend: Supabase

Base de datos: PostgreSQL

Frontend: PWA Web

Arquitectura: Multi-tenant

Mercado inicial: Venezuela

Moneda inicial: USD

Timezone inicial: America/Caracas

1. OBJETIVO DEL PROYECTO

ParkingVE será una PWA SaaS orientada principalmente a estacionamientos pequeños y medianos que:

- no poseen sistemas de gestión;
- utilizan procesos manuales;
- no desean adquirir sistemas costosos;
- necesitan controlar entradas y salidas;
- necesitan registrar vehículos;
- necesitan administrar su clientela;
- necesitan calcular tarifas;
- necesitan gestionar pagos;
- necesitan recibir comprobantes de Pago Móvil;
- necesitan controlar caja;
- necesitan consultar operaciones e históricos.

El sistema debe ser sencillo para el operador y rápido de utilizar.

El objetivo NO es construir inicialmente un sistema complejo de estacionamiento con barreras, lectores de matrícula, sensores, cámaras o hardware especializado.

El objetivo inicial es:

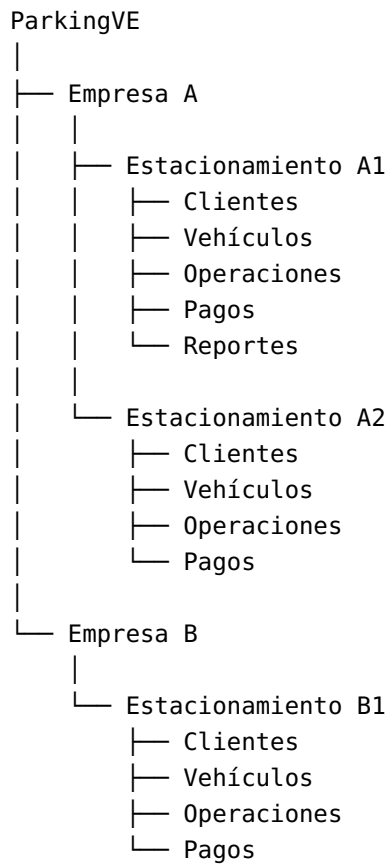
Digitalizar el proceso administrativo de un estacionamiento pequeño utilizando una PWA sencilla.

2. MODELO COMERCIAL

ParkingVE será desarrollado como SaaS.

Cada empresa que utilice ParkingVE tendrá su propio espacio independiente.

Conceptualmente:



3. REGLA FUNDAMENTAL DE AISLAMIENTO

Una empresa nunca puede acceder a los datos de otra empresa.

Un usuario solamente puede acceder a los estacionamientos a los que pertenece.

La seguridad debe implementarse mediante:

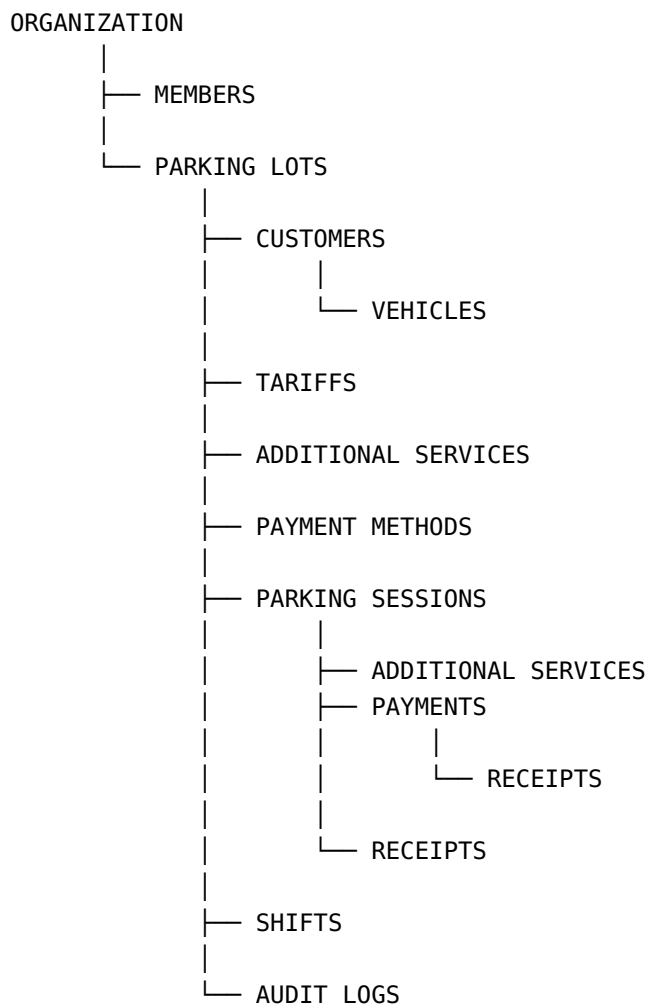
```
Supabase Auth
+
PostgreSQL
+
RLS
+
roles
+
tenant_id / organization_id
```

+
parking_lot_id

Nunca depender exclusivamente del frontend.

4. JERARQUÍA DEL SISTEMA

La jerarquía definitiva será:



5. ORGANIZATIONS

Una `organization` representa una empresa/cliente de ParkingVE.

Ejemplo:

Estacionamientos Caracas 2026 C.A.

Puede tener:

Estacionamiento Chacao
Estacionamiento Altamira
Estacionamiento Las Mercedes

6. TABLA ORGANIZATIONS

```
create table public.organizations (  
  id uuid primary key default gen_random_uuid(),  
  
  name text not null,  
  
  legal_name text,  
  
  identification_number text,  
  
  phone text,  
  
  email text,  
  
  address text,  
  
  country_code text not null default 'VE',  
  
  currency_code text not null default 'USD',  
  
  timezone text not null default 'America/Caracas',  
  
  is_active boolean not null default true,  
  
  created_at timestamptz not null default now(),  
  
  updated_at timestamptz not null default now()  
);
```

7. PROFILES

Relaciona el usuario de Supabase Auth con los datos del usuario.

```
create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,

  first_name text,

  last_name text,

  phone text,

  is_active boolean not null default true,

  created_at timestampz not null default now(),

  updated_at timestampz not null default now()
);
```

8. ORGANIZATION_MEMBERS

Relaciona usuarios con empresas.

```
create type public.organization_role as enum (
  'OWNER',
  'ADMIN',
  'OPERATOR'
);
```

```
create table public.organization_members (
  id uuid primary key default gen_random_uuid(),

  organization_id uuid not null
    references public.organizations(id)
    on delete cascade,

  user_id uuid not null
    references public.profiles(id)
    on delete cascade,

  role public.organization_role not null default 'OPERATOR',

  is_active boolean not null default true,

  created_at timestampz not null default now(),

  unique (organization_id, user_id)
);
```

9. PARKING_LOTS

Representa cada estacionamiento físico.

```
create table public.parking_lots (  
  id uuid primary key default gen_random_uuid(),  
  
  organization_id uuid not null  
    references public.organizations(id)  
    on delete cascade,  
  
  name text not null,  
  
  address text,  
  
  phone text,  
  
  email text,  
  
  timezone text not null default 'America/Caracas',  
  
  currency_code text not null default 'USD',  
  
  is_active boolean not null default true,  
  
  created_at timestamptz not null default now(),  
  
  updated_at timestamptz not null default now()  
);
```

10. PARKING_LOT_MEMBERS

Permite determinar qué usuarios pueden trabajar en cada estacionamiento.

```
create table public.parking_lot_members (  
  id uuid primary key default gen_random_uuid(),  
  
  parking_lot_id uuid not null  
    references public.parking_lots(id)  
    on delete cascade,  
  
  user_id uuid not null  
    references public.profiles(id)  
    on delete cascade,
```

```
created_at timestamptz not null default now(),  
  
unique (parking_lot_id, user_id)  
);
```

El rol se hereda inicialmente desde `organization_members`.

En una futura versión se podrá permitir un rol específico por estacionamiento.

11. CUSTOMERS

Esta tabla es fundamental.

Cada estacionamiento tendrá su propia base de clientes.

```
create table public.customers (  
  id uuid primary key default gen_random_uuid(),  
  
  parking_lot_id uuid not null  
    references public.parking_lots(id)  
    on delete cascade,  
  
  first_name text,  
  
  last_name text,  
  
  phone text,  
  
  email text,  
  
  identification_number text,  
  
  notes text,  
  
  is_active boolean not null default true,  
  
  created_at timestamptz not null default now(),  
  
  updated_at timestamptz not null default now()  
);
```

12. CLIENTES INDEPENDIENTES POR ESTACIONAMIENTO

Un cliente puede existir en varios estacionamientos.

Ejemplo:

```
Parking Chacao
Juan Pérez

Parking Altamira
Juan Pérez
```

Son registros independientes.

No crear inicialmente una base global de clientes para todos los estacionamientos.

La propiedad será:

```
customers.parking_lot_id
```

13. VEHICLES

Los vehículos pertenecen a un cliente.

```
create type public.vehicle_type as enum (
  'CAR',
  'MOTORCYCLE'
);
```

```
create table public.vehicles (
  id uuid primary key default gen_random_uuid(),

  parking_lot_id uuid not null
    references public.parking_lots(id)
    on delete cascade,

  customer_id uuid
    references public.customers(id)
    on delete set null,

  plate text not null,
```

```
vehicle_type public.vehicle_type not null,  
  
brand text,  
  
model text,  
  
color text,  
  
notes text,  
  
created_at timestampz not null default now(),  
  
updated_at timestampz not null default now(),  
  
unique (parking_lot_id, plate)  
);
```

14. MATRÍCULA

Antes de almacenar una matrícula:

ABC - 123

normalizar:

ABC123

La búsqueda también debe normalizar el valor introducido.

Esto evita duplicados causados por:

ABC123
ABC - 123
abc123
abc - 123

15. TARIFFS

Las tarifas son configurables por estacionamiento.

```

create table public.tariffs (
  id uuid primary key default gen_random_uuid(),

  parking_lot_id uuid not null
    references public.parking_lots(id)
    on delete cascade,

  vehicle_type public.vehicle_type not null,

  daily_price numeric(12,2) not null default 0,

  lost_ticket_price numeric(12,2) not null default 0,

  currency_code text not null default 'USD',

  is_active boolean not null default true,

  created_at timestamptz not null default now(),

  updated_at timestamptz not null default now(),

  unique (parking_lot_id, vehicle_type)
);

```

16. TARIFAS INICIALES

El sistema debe soportar como mínimo:

CARRO

- Tarifa diaria
- Ticket perdido

MOTO

- Tarifa diaria
- Ticket perdido

Los precios son editables.

No colocar precios directamente en el frontend.

17. ADDITIONAL_SERVICES

Servicios adicionales.

Ejemplo:

```
Casco  
Lavado  
Servicio especial  
Otro
```

```
create table public.additional_services (  
  id uuid primary key default gen_random_uuid(),  
  
  parking_lot_id uuid not null  
    references public.parking_lots(id)  
    on delete cascade,  
  
  name text not null,  
  
  price numeric(12,2) not null,  
  
  currency_code text not null default 'USD',  
  
  is_active boolean not null default true,  
  
  created_at timestamptz not null default now(),  
  
  updated_at timestamptz not null default now()  
);
```

18. PAYMENT_METHODS

Métodos de pago configurados por cada estacionamiento.

```
create type public.payment_method_type as enum (  
  'CASH',  
  'POS',  
  'MOBILE_PAYMENT',  
  'BANK_TRANSFER'  
);
```

```
create table public.payment_methods (  
  id uuid primary key default gen_random_uuid(),  
  
  parking_lot_id uuid not null  
    references public.parking_lots(id)  
    on delete cascade,
```

```

method_type public.payment_method_type not null,

display_name text not null,

bank_name text,

account_holder text,

identification_number text,

phone_number text,

instructions text,

is_active boolean not null default true,

created_at timestamptz not null default now(),

updated_at timestamptz not null default now()
);

```

19. PARKING_SESSIONS

Tabla principal de entradas y salidas.

```

create type public.session_status as enum (
    'ACTIVE',
    'PAYMENT_PENDING',
    'COMPLETED',
    'CANCELLED'
);

```

```

create table public.parking_sessions (
    id uuid primary key default gen_random_uuid(),

    parking_lot_id uuid not null
        references public.parking_lots(id)
        on delete restrict,

    customer_id uuid
        references public.customers(id)
        on delete set null,

    vehicle_id uuid not null
        references public.vehicles(id)

```

```

    on delete restrict,

plate_snapshot text not null,

vehicle_type_snapshot public.vehicle_type not null,

entry_at timestamptz not null default now(),

exit_at timestamptz,

status public.session_status not null default 'ACTIVE',

lost_ticket boolean not null default false,

parking_fee numeric(12,2) not null default 0,

lost_ticket_fee numeric(12,2) not null default 0,

additional_services_total numeric(12,2) not null default 0,

total_amount numeric(12,2) not null default 0,

currency_code text not null default 'USD',

customer_email text,

customer_phone text,

entry_operator_id uuid
    references public.profiles(id)
    on delete set null,

exit_operator_id uuid
    references public.profiles(id)
    on delete set null,

entry_notes text,

exit_notes text,

created_at timestamptz not null default now(),

updated_at timestamptz not null default now()
);

```

20. SNAPSHOT DE INFORMACIÓN

La sesión conservará los datos históricos.

Por ejemplo:

```
plate_snapshot  
vehicle_type_snapshot  
parking_fee  
lost_ticket_fee  
total_amount  
currency_code
```

Esto evita que cambiar una tarifa posteriormente modifique operaciones históricas.

21. SESSION_ADDITIONAL_SERVICES

```
create table public.session_additional_services (  
  id uuid primary key default gen_random_uuid(),  
  
  session_id uuid not null  
    references public.parking_sessions(id)  
    on delete cascade,  
  
  additional_service_id uuid  
    references public.additional_services(id)  
    on delete set null,  
  
  service_name_snapshot text not null,  
  
  unit_price numeric(12,2) not null,  
  
  quantity integer not null default 1,  
  
  total_price numeric(12,2) not null,  
  
  currency_code text not null default 'USD',  
  
  created_at timestamptz not null default now()  
);
```

22. PAYMENTS

```
create type public.payment_status as enum (  
  'PENDING',  
  'RECEIPT_REQUIRED',  
  'UNDER_REVIEW',
```

```
'APPROVED',  
'REJECTED',  
'CANCELLED'  
);
```

```
create table public.payments (  
  id uuid primary key default gen_random_uuid(),  
  
  parking_lot_id uuid not null  
    references public.parking_lots(id)  
    on delete restrict,  
  
  session_id uuid not null  
    references public.parking_sessions(id)  
    on delete restrict,  
  
  payment_method_id uuid  
    references public.payment_methods(id)  
    on delete set null,  
  
  method_type public.payment_method_type not null,  
  
  amount numeric(12,2) not null,  
  
  currency_code text not null default 'USD',  
  
  status public.payment_status not null default 'PENDING',  
  
  reference_number text,  
  
  customer_email text,  
  
  customer_phone text,  
  
  paid_at timestampz,  
  
  reviewed_at timestampz,  
  
  reviewed_by uuid  
    references public.profiles(id)  
    on delete set null,  
  
  rejection_reason text,  
  
  notes text,  
  
  created_at timestampz not null default now(),
```

```
updated_at timestampz not null default now()
);
```

23. PAYMENT_RECEIPTS

El cliente podrá adjuntar el comprobante.

```
create type public.receipt_status as enum (
    'RECEIVED',
    'APPROVED',
    'REJECTED'
);
```

```
create table public.payment_receipts (
    id uuid primary key default gen_random_uuid(),

    payment_id uuid not null
        references public.payments(id)
        on delete cascade,

    storage_path text not null,

    original_filename text,

    mime_type text,

    file_size_bytes bigint,

    status public.receipt_status not null default 'RECEIVED',

    uploaded_at timestampz not null default now(),

    reviewed_at timestampz,

    reviewed_by uuid
        references public.profiles(id)
        on delete set null,

    rejection_reason text
);
```

24. STORAGE

Crear bucket privado:

```
payment-receipts
```

Estructura:

```
payment-receipts/  
  organization_id/  
    parking_lot_id/  
      payment_id/  
        uuid.jpg
```

Los comprobantes nunca serán públicos.

25. PUBLIC_PAYMENT_LINKS

Para que el cliente pueda pagar desde su teléfono.

```
create type public.public_link_type as enum (  
  'PAYMENT',  
  'RECEIPT'  
);
```

```
create table public.public_payment_links (  
  id uuid primary key default gen_random_uuid(),  
  
  parking_lot_id uuid not null  
    references public.parking_lots(id)  
    on delete cascade,  
  
  session_id uuid not null  
    references public.parking_sessions(id)  
    on delete cascade,  
  
  payment_id uuid  
    references public.payments(id)  
    on delete cascade,  
  
  link_type public.public_link_type not null,  
  
  token_hash text not null unique,
```

```
expires_at timestamptz,  
  
is_active boolean not null default true,  
  
created_at timestamptz not null default now()  
);
```

26. FLUJO DE PAGO

El flujo será:

```
Operador registra salida  
↓  
Sistema calcula total  
↓  
Cliente indica email  
↓  
Sistema muestra métodos de pago  
↓  
Pago Móvil  
↓  
Cliente realiza pago  
↓  
Cliente adjunta comprobante  
↓  
ParkingVE registra comprobante  
↓  
Operador revisa  
↓  
APPROVED  
↓  
Se permite completar salida  
↓  
Se genera recibo
```

27. ENVÍO DE INFORMACIÓN AL CLIENTE

MVP:

El sistema debe preparar el flujo para enviar:

```
correo electrónico
```

con:

```
Estacionamiento
Matrícula
Monto
Datos de Pago Móvil
Enlace para adjuntar comprobante
```

La integración de proveedor de correo se implementará posteriormente.

La base de datos debe conservar:

```
customer_email
```

en la operación.

28. RECEIPTS

Recibo histórico.

```
create table public.receipts (
  id uuid primary key default gen_random_uuid(),

  parking_lot_id uuid not null
    references public.parking_lots(id)
    on delete restrict,

  session_id uuid not null unique
    references public.parking_sessions(id)
    on delete restrict,

  payment_id uuid
    references public.payments(id)
    on delete set null,

  receipt_number bigint generated always as identity,

  plate_snapshot text not null,

  vehicle_type_snapshot public.vehicle_type not null,

  entry_at timestamptz not null,

  exit_at timestamptz,

  subtotal numeric(12,2) not null,
```

```
total_amount numeric(12,2) not null,  
  
currency_code text not null,  
  
payment_method public.payment_method_type,  
  
created_at timestamptz not null default now()  
);
```

29. SHIFTS

Control de caja por operador.

```
create type public.shift_status as enum (  
    'OPEN',  
    'CLOSED'  
);
```

```
create table public.shifts (  
    id uuid primary key default gen_random_uuid(),  
  
    parking_lot_id uuid not null  
        references public.parking_lots(id)  
        on delete restrict,  
  
    operator_id uuid not null  
        references public.profiles(id)  
        on delete restrict,  
  
    status public.shift_status not null default 'OPEN',  
  
    opened_at timestamptz not null default now(),  
  
    closed_at timestamptz,  
  
    opening_cash numeric(12,2) not null default 0,  
  
    expected_cash numeric(12,2) not null default 0,  
  
    declared_cash numeric(12,2),  
  
    cash_difference numeric(12,2),  
  
    notes text,
```

```
    created_at timestamptz not null default now()
);
```

30. AUDIT_LOGS

```
create table public.audit_logs (
    id uuid primary key default gen_random_uuid(),

    organization_id uuid
        references public.organizations(id)
        on delete cascade,

    parking_lot_id uuid
        references public.parking_lots(id)
        on delete cascade,

    user_id uuid
        references public.profiles(id)
        on delete set null,

    action text not null,

    entity_type text,

    entity_id uuid,

    metadata jsonb,

    ip_address inet,

    user_agent text,

    created_at timestamptz not null default now()
);
```

31. ÍNDICES

Crear como mínimo:

```
create index idx_org_members_user
on public.organization_members(user_id);

create index idx_org_members_org
on public.organization_members(organization_id);
```

```
create index idx_parking_lots_org
on public.parking_lots(organization_id);

create index idx_parking_lot_members_user
on public.parking_lot_members(user_id);

create index idx_customers_parking
on public.customers(parking_lot_id);

create index idx_customers_email
on public.customers(parking_lot_id, email);

create index idx_vehicles_parking_plate
on public.vehicles(parking_lot_id, plate);

create index idx_vehicles_customer
on public.vehicles(customer_id);

create index idx_sessions_parking_status
on public.parking_sessions(parking_lot_id, status);

create index idx_sessions_plate
on public.parking_sessions(parking_lot_id, plate_snapshot);

create index idx_sessions_entry
on public.parking_sessions(parking_lot_id, entry_at);

create index idx_payments_session
on public.payments(session_id);

create index idx_payments_status
on public.payments(parking_lot_id, status);

create index idx_receipts_payment
on public.payment_receipts(payment_id);

create index idx_public_links_token
on public.public_payment_links(token_hash);

create index idx_audit_logs_parking_date
on public.audit_logs(parking_lot_id, created_at desc);
```

32. SESIÓN ACTIVA DUPLICADA

No permitir que un vehículo tenga simultáneamente dos sesiones activas en el mismo estacionamiento.

```
create unique index idx_one_active_session_per_vehicle
on public.parking_sessions(parking_lot_id, vehicle_id)
where status in ('ACTIVE', 'PAYMENT_PENDING');
```

33. REGLAS DE NEGOCIO

Entrada

Debe validar:

```
usuario autorizado
parking correcto
matrícula válida
vehículo válido
no existe sesión activa
```

Después:

```
buscar cliente
↓
buscar vehículo
↓
si no existe, crear
↓
crear sesión
```

34. CLIENTE NUEVO

Si la matrícula no existe:

```
ABC123
```

el operador podrá:

```
Crear cliente
```

con:

Nombre
Apellido
Teléfono
Email
Identificación

Después:

Crear vehículo

35. CLIENTE EXISTENTE

Si:

ABC123

ya pertenece a:

Juan Pérez

mostrar:

Juan Pérez
ABC123
Toyota Corolla

El operador no tendrá que volver a introducir los datos.

36. SALIDA

Al seleccionar:

ABC123

mostrar:

Cliente
Vehículo

Hora entrada
Tiempo estacionado
Tarifa
Adicionales
Ticket perdido
Total

37. TICKET PERDIDO

Si el operador selecciona:

Ticket perdido

utilizar:

lost_ticket_price

configurado para el tipo de vehículo.

El sistema debe mostrar claramente el concepto aplicado.

38. ADICIONALES

Ejemplo:

Casco
\$1

El operador puede añadir:

Casco x 1

El total se recalcula.

39. CÁLCULO DEL TOTAL

Conceptualmente:

```
Total =  
tarifa estacionamiento  
+  
ticket perdido  
+  
servicios adicionales
```

La tarifa debe ser calculada/verificada en backend.

No confiar en un total enviado únicamente desde el frontend.

40. APROBACIÓN DEL PAGO

Un pago con comprobante debe pasar:

```
PENDING  
↓  
RECEIPT_REQUIRED  
↓  
UNDER_REVIEW  
↓  
APPROVED
```

o:

```
UNDER_REVIEW  
↓  
REJECTED
```

Si es rechazado:

```
rejection_reason
```

es obligatorio.

41. SALIDA COMPLETADA

No permitir:

```
status = COMPLETED
```

si el pago requerido no ha sido aprobado.

La operación debe ejecutarse como una operación segura/transaccional.

42. RECIBO

Después de:

```
payment = APPROVED
```

y:

```
session = COMPLETED
```

crear:

```
receipt
```

con información histórica.

43. ROLES

OWNER

Acceso:

```
Empresa  
Estacionamientos  
Usuarios  
Clientes  
Vehículos  
Tarifas  
Servicios  
Pagos  
Caja  
Reportes  
Auditoría
```

ADMIN

Acceso:

Estacionamientos
Usuarios
Clientes
Vehículos
Tarifas
Servicios
Pagos
Caja
Reportes

OPERATOR

Acceso:

Clientes
Vehículos
Entrada
Salida
Pagos
Comprobantes
Turno

No puede modificar configuraciones críticas.

44. RLS

Activar RLS en todas las tablas públicas.

La lógica general:

```
auth.uid()
  ↓
organization_members
  ↓
organization_id
  ↓
parking_lots
  ↓
parking_lot_id
```

Un usuario solamente podrá acceder a los recursos de las organizaciones/estacionamientos donde tenga membresía válida.

45. ACCESO PÚBLICO

El cliente que paga no necesita una cuenta ParkingVE.

Utilizará:

```
public_payment_links
```

con token aleatorio.

El cliente solamente podrá acceder a la operación asociada al token.

Nunca exponer:

```
session UUID  
payment UUID  
customer UUID
```

como mecanismo de autorización pública.

46. SUPABASE STORAGE

Bucket:

```
payment-receipts
```

Configuración:

```
PRIVATE
```

Los operadores autorizados recibirán URLs temporales para visualizar comprobantes.

47. SEGURIDAD

Nunca colocar claves privilegiadas en:

```
React  
browser
```

```
frontend
GitHub
```

La `service_role` solamente podrá utilizarse en backend/server-side cuando sea necesario.

48. ELIMINACIÓN

No eliminar físicamente:

```
parking_sessions
payments
payment_receipts
receipts
audit_logs
```

Mantener históricos.

Para configuraciones:

```
is_active = false
```

49. MIGRACIONES

Antigravity debe crear:

```
supabase/
├─ migrations/
│   ├── 001_extensions.sql
│   ├── 002_enums.sql
│   ├── 003_profiles.sql
│   ├── 004_organizations.sql
│   ├── 005_organization_members.sql
│   ├── 006_parking_lots.sql
│   ├── 007_parking_lot_members.sql
│   ├── 008_customers.sql
│   ├── 009_vehicles.sql
│   ├── 010_tariffs.sql
│   ├── 011_additional_services.sql
│   ├── 012_payment_methods.sql
│   ├── 013_parking_sessions.sql
│   ├── 014_session_services.sql
│   ├── 015_payments.sql
│   └── 016_payment_receipts.sql
```

```
|— 017_public_links.sql
|— 018_receipts.sql
|— 019_shifts.sql
|— 020_audit_logs.sql
|— 021_indexes.sql
|— 022_functions.sql
|— 023_rls.sql
```

50. ORDEN GENERAL DE DESARROLLO

FASE 1 — Preparación

```
Crear repositorio
Configurar proyecto
Configurar Supabase
Configurar variables de entorno
Configurar PWA
```

FASE 2 — Base de datos

```
ENUMS
↓
Organizations
↓
Users
↓
Members
↓
Parking lots
↓
Customers
↓
Vehicles
↓
Tariffs
↓
Services
↓
Payments
↓
Sessions
↓
Receipts
↓
```

Shifts
↓
Audit
↓
RLS

No comenzar las pantallas principales hasta completar esta fase.

51. FASE 3 — AUTENTICACIÓN

Implementar:

Login
Logout
Password recovery
Session management

Después:

Usuario
↓
Organization
↓
Parking lot
↓
Rol

52. FASE 4 — CONFIGURACIÓN DEL ESTACIONAMIENTO

Crear interfaz:

Mi estacionamiento

Permitir:

Nombre
Dirección
Teléfono
Email

Moneda
Timezone

53. FASE 5 — TARIFAS

Pantalla:

Configuración
└─ Tarifas

Mostrar:

CARRO
Tarifa diaria: \$__
Ticket perdido: \$__

MOTO
Tarifa diaria: \$__
Ticket perdido: \$__

Todos los valores editables.

54. FASE 6 — SERVICIOS ADICIONALES

Pantalla:

Configuración
└─ Servicios

Ejemplo:

Casco	\$1
Lavado	\$5
Otro	\$2

Permitir:

Crear
Editar

Activar
Desactivar

55. FASE 7 — CLIENTES

Pantalla:

Clientes

Funciones:

Buscar
Crear
Editar
Ver historial
Ver vehículos

Búsqueda por:

nombre
teléfono
email
matrícula

56. FASE 8 — ENTRADA

Pantalla extremadamente rápida.

REGISTRAR ENTRADA

Campo principal:

MATRÍCULA

Después:

CARRO
MOTO

Si cliente existe:

mostrar cliente

Si no:

crear cliente

Finalmente:

REGISTRAR ENTRADA

57. FASE 9 — VEHÍCULOS ACTIVOS

Dashboard:

Vehículos actualmente dentro

Mostrar:

Matrícula
Cliente
Tipo
Hora entrada
Tiempo

58. FASE 10 — SALIDA

Buscar:

Matrícula

Mostrar operación.

Después:

Tarifa
Ticket perdido

Adicionales
TOTAL

Botón:

CONTINUAR AL PAGO

59. FASE 11 — PAGO

Mostrar:

TOTAL A PAGAR

Métodos:

Efectivo
Punto de venta
Pago Móvil
Transferencia

Para Pago Móvil:

Banco
Teléfono
Cédula/RIF
Titular
Monto

60. FASE 12 — COMPROBANTE

Cliente puede:

Subir imagen

Formatos iniciales:

JPG
JPEG
PNG

WEBP
PDF

Validar:

tipo
tamaño

61. FASE 13 — REVISIÓN

Dashboard:

Pagos pendientes

Mostrar:

Cliente
Matrícula
Monto
Método
Fecha
Comprobante

Acciones:

APROBAR
RECHAZAR

62. FASE 14 — RECIBO

Después de aprobación:

Pago aprobado
↓
Salida completada
↓
Recibo

El recibo debe poder:

visualizarse
descargarse
compartirse

La generación PDF puede implementarse después del MVP funcional.

63. FASE 15 — CAJA

Crear:

Abrir turno
Cerrar turno

Mostrar:

Efectivo
POS
Pago Móvil
Transferencias
Total

64. FASE 16 — REPORTES

MVP:

Ingresos del día
Ingresos del período
Cantidad de vehículos
Cantidad de operaciones
Pagos pendientes

Filtros:

Hoy
Ayer
Últimos 7 días
Mes
Personalizado

65. FASE 17 — AUDITORÍA

Registrar:

login
entrada
salida
pago
aprobación
rechazo
cambio de tarifa
cambio de servicio
creación de usuario
cierre de turno

66. DASHBOARD MVP

El dashboard principal debe mostrar:

VEHÍCULOS DENTRO 27
INGRESOS HOY \$125
PAGOS PENDIENTES 3
OPERACIONES HOY 42

Acciones rápidas:

[+ ENTRADA]

[SALIDA]

[CLIENTES]

[PAGOS]

67. PRINCIPIO UX

El operador puede estar atendiendo vehículos.

Por tanto:

```
menos clics
menos campos
menos pantallas
más velocidad
```

La función principal debe ser:

```
Matrícula
↓
Entrada
```

y:

```
Matrícula
↓
Salida
↓
Pago
```

68. RESPONSIVE

La aplicación debe funcionar correctamente en:

```
Desktop
Tablet
iPhone
Android
```

Prioridad:

```
Desktop
+
móvil
```

porque el operador puede utilizar un teléfono/tablet en el estacionamiento.

69. PWA

Implementar:

```
manifest
service worker
installable app
responsive UI
offline shell
```

El MVP puede requerir conexión para operaciones críticas.

No implementar todavía sincronización offline compleja.

70. TECNOLOGÍAS

Utilizar:

```
Frontend:
React / Next.js según configuración actual del proyecto

Backend:
Supabase

Database:
PostgreSQL

Authentication:
Supabase Auth

Storage:
Supabase Storage

PWA:
Service Worker + Web App Manifest
```

No introducir tecnologías adicionales sin necesidad.

71. REGLA PARA ANTIGRAVITY

Antes de escribir código:

1. Analizar estructura actual.
2. Identificar archivos existentes.
3. No sobrescribir código útil.
4. Revisar dependencias.
5. Crear plan de implementación.
6. Ejecutar una fase.
7. Probar.
8. Corregir.
9. Documentar.
10. Continuar.

No realizar toda la aplicación de una sola vez.

72. ORDEN DE EJECUCIÓN INICIAL

Antigravity debe comenzar exactamente así:

PASO 1

Analizar proyecto actual.

PASO 2

Verificar configuración de Supabase.

PASO 3

Crear estructura de migraciones.

PASO 4

Crear ENUMS.

PASO 5

Crear Organizations.

PASO 6

Crear Profiles.

PASO 7

Crear Organization Members.

PASO 8

Crear Parking Lots.

PASO 9

Crear Parking Lot Members.

PASO 10

Crear Customers.

PASO 11
Crear Vehicles.

PASO 12
Crear Tariffs.

PASO 13
Crear Additional Services.

PASO 14
Crear Payment Methods.

PASO 15
Crear Parking Sessions.

PASO 16
Crear Session Additional Services.

PASO 17
Crear Payments.

PASO 18
Crear Payment Receipts.

PASO 19
Crear Public Payment Links.

PASO 20
Crear Receipts.

PASO 21
Crear Shifts.

PASO 22
Crear Audit Logs.

PASO 23
Crear índices.

PASO 24
Crear funciones.

PASO 25
Crear RLS.

PASO 26
Crear políticas.

PASO 27
Ejecutar tests.

73. PRIMER PROMPT PARA ANTIGRAVITY

Copiar exactamente:

INICIO DEL DESARROLLO PARKINGVE

Vas a desarrollar ParkingVE siguiendo estrictamente el Documento Maestro Técnico v2.0.

NO empieces todavía por las pantallas.

Primero analiza el proyecto actual y determina:

1. Framework utilizado.
2. Estructura de carpetas.
3. Dependencias.
4. Configuración actual de Supabase.
5. Variables de entorno existentes.
6. Si existen migraciones.
7. Si existe autenticación.
8. Si existe código reutilizable.

Después crea un plan de ejecución para la **FASE 1 — Preparación** y **FASE 2 — Base de datos**.

La arquitectura de datos obligatoria es:

```
organizations → parking_lots → customers → vehicles →  
parking_sessions → payments → receipts
```

Los usuarios estarán relacionados mediante:

```
profiles → organization_members → parking_lot_members
```

Cada empresa debe estar completamente aislada de las demás.

Implementa PostgreSQL mediante migraciones Supabase.

Implementa RLS desde el inicio.

No coloques lógica crítica exclusivamente en el frontend.

No utilices datos mock como sustituto de la base de datos real.

No avances a la interfaz de Entrada/Salida hasta que:

- las migraciones estén funcionando;
- las tablas estén creadas;
- las relaciones estén verificadas;
- los índices estén creados;
- RLS esté activo;
- las políticas hayan sido probadas;
- los datos de una empresa no sean visibles para otra;
- exista una prueba de aislamiento multi-tenant.

Trabaja fase por fase.

Después de cada fase:

1. Ejecuta pruebas.
2. Corrige errores.
3. Explica qué fue implementado.
4. Indica qué archivos fueron creados/modificados.
5. Espera la siguiente instrucción.

NO avances automáticamente a la siguiente fase.

Comienza ahora con el análisis del proyecto.

74. DEFINICIÓN DEL MVP

El MVP estará completo cuando un estacionamiento pueda:

1. Crear usuarios
2. Configurar estacionamiento
3. Configurar tarifas
4. Configurar servicios
5. Configurar Pago Móvil
6. Crear clientes
7. Registrar vehículos
8. Registrar entrada
9. Registrar salida
10. Calcular tarifa
11. Registrar pago
12. Mostrar datos de Pago Móvil
13. Recibir comprobante
14. Aprobar/rechazar comprobante
15. Completar salida
16. Generar recibo
17. Controlar turno/caja

- 18. Consultar historial
- 19. Consultar reportes

75. FUTURAS VERSIONES

NO implementar todavía:

- Reconocimiento automático de matrículas
- Cámaras
- Barreras automáticas
- Sensores
- Control de puestos individuales
- GPS
- Facturación fiscal electrónica
- Integraciones bancarias automáticas
- IA
- Chatbot
- WhatsApp automático
- App móvil nativa

Estas funcionalidades podrán incorporarse posteriormente.

76. VISIÓN FUTURA

La arquitectura permitirá posteriormente:

- ParkingVE
 - |
 - |— SaaS
 - |— API
 - |— App móvil
 - |— WhatsApp
 - |— QR
 - |— cámaras
 - |— reconocimiento de matrículas
 - |— automatización
 - |— múltiples estacionamientos

sin reconstruir el núcleo de datos.

77. CRITERIO PRINCIPAL DEL PRODUCTO

ParkingVE debe resolver una necesidad concreta:

Que un estacionamiento pequeño pueda controlar quién entra, quién sale, cuánto debe pagar, cómo paga, comprobar el pago y conservar el historial sin necesitar un sistema costoso o complejo.

La experiencia debe ser:

RÁPIDA
SIMPLE
CLARA
ECONÓMICA
SEGURA

FIN DEL DOCUMENTO MAESTRO TÉCNICO v2.0

PARKINGVE